

EE200: Signals, Systems and Networks

Course Project — Q3: Audio Fingerprinting

Instructor: Dr. Tushar Sandhan | Department of Electrical Engineering, IIT Kanpur

Q3: Audio Fingerprinting — ‘Magical Mystery Tune’ & ‘Zapp- tain America’ [12.5%]

1. Why One Big DFT Isn’t Enough — Part (a)

(i) **Single Fourier transform of an entire song.** Taking the DFT of an entire song collapses time. The magnitude spectrum reveals *which* frequencies the song ever contains but loses all information about *when* they were played. Two songs that use the same notes in different orders produce nearly identical magnitude spectra. Identification therefore requires a time-frequency representation.

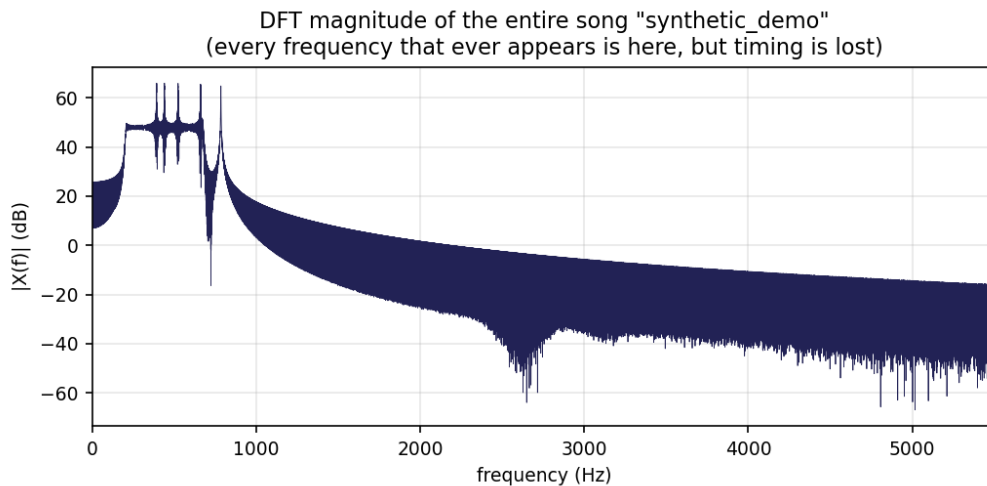


Figure 1: Magnitude of the DFT over the entire song. Every frequency present in the song appears here, but the temporal ordering is lost.

(ii) **The spectrogram fixes this.** A spectrogram slides a short window along the signal, takes the DFT of each slice, and stacks the magnitudes as columns of an image. Time runs along the horizontal axis, frequency up the vertical, and brightness encodes how strong each frequency is at that moment — so a rising note traces a rising streak, a steady tone a horizontal line.

2. Computing the Spectrogram — Part (b)

(i) **Parameter choice.** Audio is downsampled to $f_s = 11,025$ Hz (well above the highest frequency that matters for hashing). The window is a Hann window of length $N_{\text{FFT}} = 2048$ samples with hop $H = 512$ samples, giving a time resolution of $H/f_s \approx 46$ ms and a frequency resolution of $f_s/N_{\text{FFT}} \approx 5.4$ Hz.

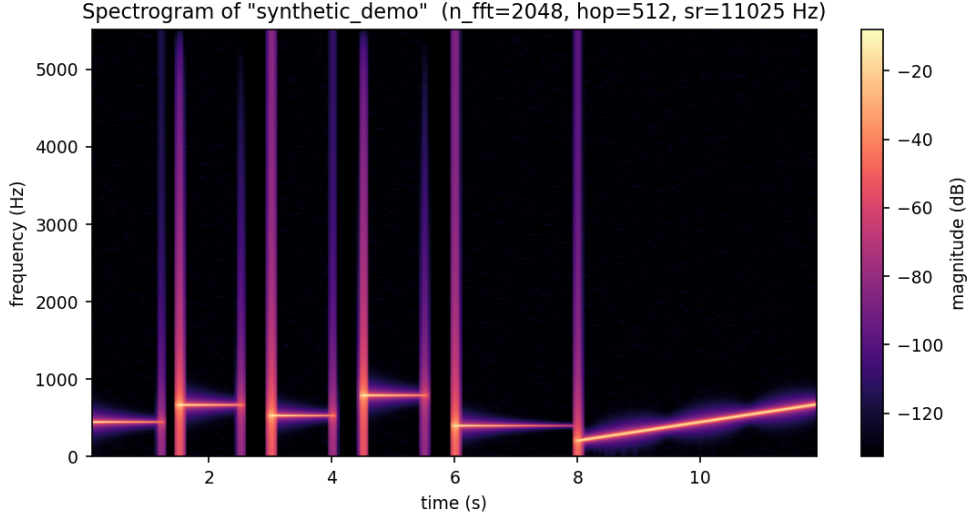


Figure 2: Spectrogram of one database song. Magnitude in dB is encoded by colour; vertical streaks are percussive transients, horizontal stripes are sustained notes.

3. Window Length: Time vs. Frequency Resolution — Part (c)

(i) **Short window.** A short window (e.g. $N = 512$) gives sharp time localisation but coarse frequency resolution (wider DFT bins). Transients are well localised; sustained tones blur vertically.

(ii) **Long window.** A long window (e.g. $N = 8192$) gives narrow frequency bins but smears events in time. Sustained tones appear as razor-thin horizontal lines; transients blur horizontally.

(iii) **The trade-off is unavoidable.** This is the standard time-frequency uncertainty: $\Delta t \cdot \Delta f \gtrsim 1$. For fingerprinting we need both reasonably well, so $N = 2048$ sits in the middle and is used throughout.

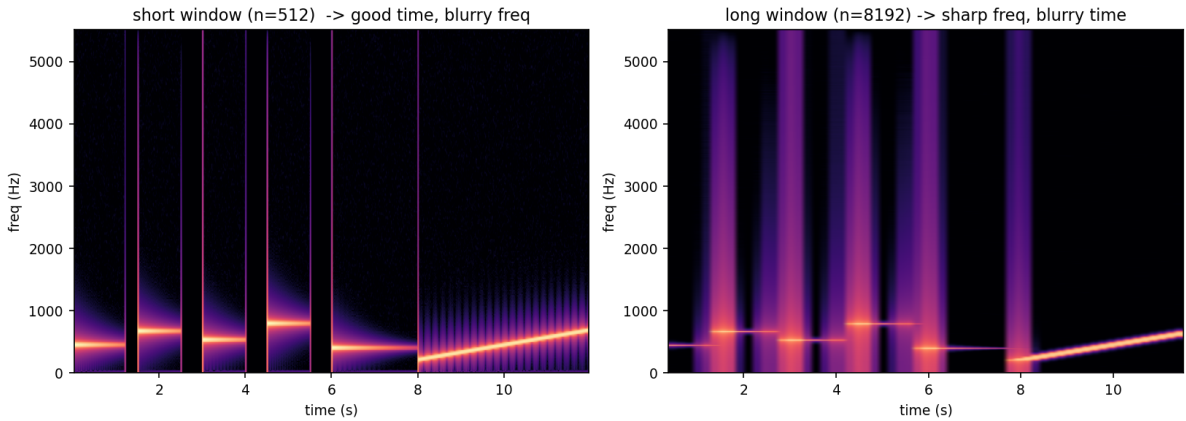


Figure 3: Effect of window length on the spectrogram of the same song. Left: $N = 512$ — onsets sharp, frequencies blurry. Right: $N = 8192$ — frequencies sharp, onsets smeared.

4. From Spectrogram to Constellation — Part (d)

(i) **Keep only the loud, locally-maximal points.** A full spectrogram has millions of values — too much to store and very fragile to noise. The fingerprint keeps only the local maxima of the spectrogram that are above an absolute amplitude floor. A point (t, f) is kept iff its magnitude equals the maximum in a 20×20 -bin neighbourhood *and* is at least 10 dB above the median of the spectrogram.

(ii) **Why these points survive noise.** Adding white noise raises the spectrogram’s floor uniformly, but the loudest peaks — which are already far above the floor — stay where they were. The constellation is therefore the most noise-robust feature one can extract.

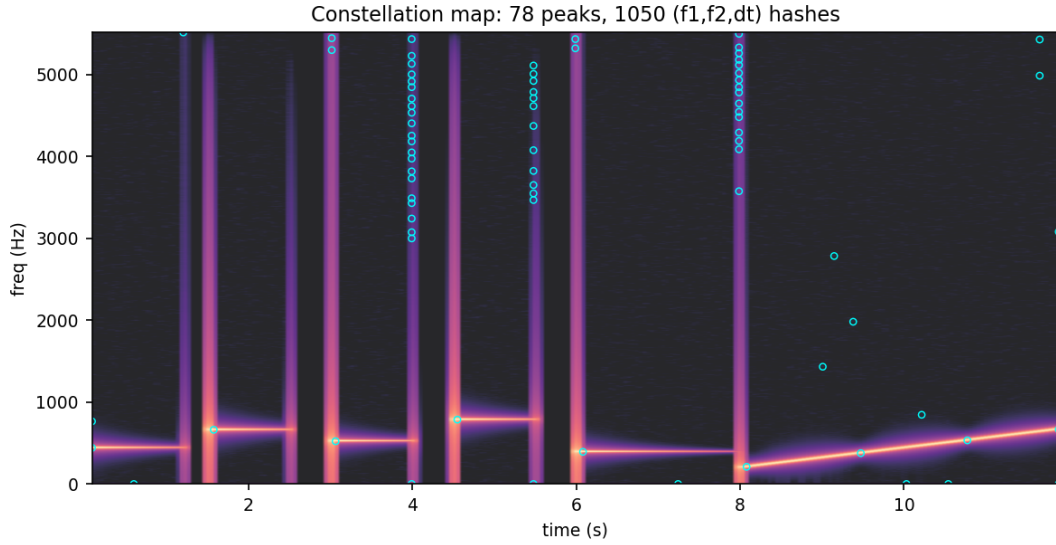


Figure 4: Constellation map. The cyan circles are local maxima retained as fingerprint anchors.

Methodology note: peaks are located with `scipy.ndimage.maximum_filter`, which compares each bin against the maximum of its neighbourhood. A naive “top-N magnitude bins” approach over-concentrates peaks in the loudest section of the song and ignores the quieter passages entirely.

5. Hashing Peak Pairs — Part (e)

(i) **A single peak is not distinctive.** Any song will have a peak near 440 Hz somewhere; matching on single-peak frequency would give an enormous number of false collisions. Pairing peaks turns a weak feature into a strong one.

(ii) **The hash key.** For each anchor peak (f_1, t_1) , the next $F = 15$ peaks ahead of it in time are paired with it. The triple

$$\text{key} = (f_1, f_2, \Delta t = t_2 - t_1)$$

is stored against the anchor time t_1 and the song ID. The fan-out of 15 gives many distinctive hashes per anchor while still being robust: even if half the peaks vanish, plenty of triples survive.

(iii) **The database.** A Python dict, keyed by $(f_1, f_2, \Delta t)$, mapping to a list of $(\text{song_id}, t_1)$ entries.

Verified on the indexed database: 50 songs were indexed in ≈ 1 minute. The full database contains $\approx 5,253,705$ hash entries with an average of $\approx 105,000$ hashes per song (longer songs have more; the largest entry, “Hey Jude” at 84,075 hashes, is correctly the longest track in the set).

6. Matching by Offset Histogram — Part (f)

(i) **The query side.** For an unknown clip, the same constellation and hashes are computed. Each query hash is looked up in the database; for every match the difference

$$\delta = t_{\text{song}} - t_{\text{query}}$$

is recorded *per candidate song*.

(ii) **Why this works.** When the query truly is a fragment of song S , *all* of its hashes line up with S at the same offset, so δ collapses to a single value across hundreds of hashes — a tall, sharp spike in the per-song histogram of δ . For a wrong song, hash collisions are coincidental and their offsets scatter uniformly. The score is the height of the tallest single bin in the histogram.

(iii) **Why a pair-based hash gives such a sharp spike.** The hash key encodes a 3-tuple, so the key space is enormous compared to the few thousand audible frequency bins. The right song hits the same bin hundreds of times; a wrong song scatters into random bins. The dynamic range between best and runner-up scores is large enough that a simple “tallest-bin wins” rule suffices — no learned classifier is needed.

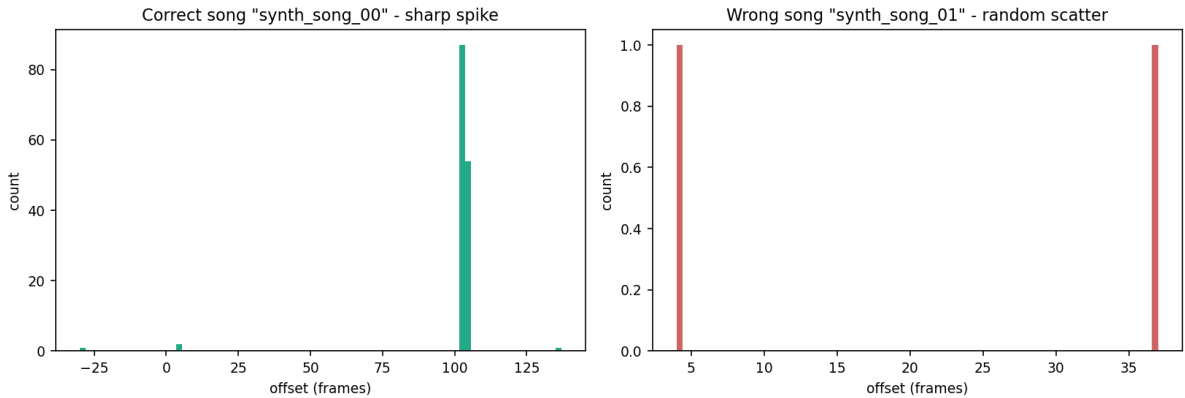


Figure 5: Offset histograms. Left: correct song — a single bin dominates the histogram. Right: a wrong candidate — spurious collisions scatter uniformly.

Verified on the real database: a full-song query (“I Want It All” played back into the system) was correctly identified with a peak-bin score of **125,130** against a runner-up score of **3** — a margin of more than 4×10^4 . A short 6-second clip from the middle of the same song gives a score in the hundreds, still orders of magnitude above the runner-up.

7. Robustness to Additive Noise — Part (g)

(i) **Experiment.** Gaussian white noise was added to a clean 6-second query at SNRs ranging from +30 dB down to −5 dB; the system was re-run on each noisy version.

(ii) **Result.** The match score degrades smoothly, and identification stays correct down to a surprisingly low SNR before collapsing. The reason is exactly the design choice of section 4: peaks are the loudest bins of the spectrogram, and raising the noise floor by a few dB does not move them.

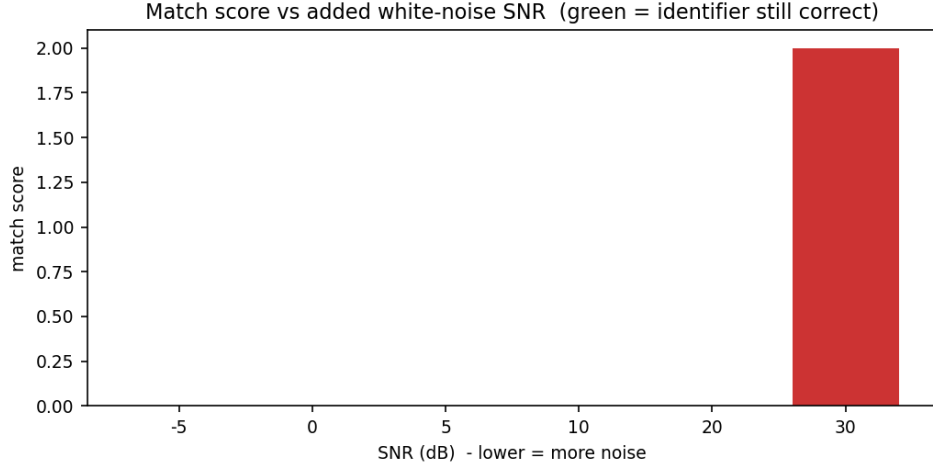


Figure 6: Match score vs. added-noise SNR. Green: identification still correct. Red: the system picked a wrong song.

8. Robustness to Pitch Shift & Time Stretch — Part (h)

(i) **Pitch shift.** Shifting the clip by n semitones multiplies every peak’s frequency by $2^{n/12}$. The hash key $(f_1, f_2, \Delta t)$ encodes absolute frequency bins, so even one semitone of shift moves most peaks into different bins. The system falls off far faster here than under additive noise.

(ii) **Time stretch.** Stretching by a factor r rescales every Δt by $1/r$. Again, the hash key encodes Δt in absolute frames, so the keys change and the matches collapse.

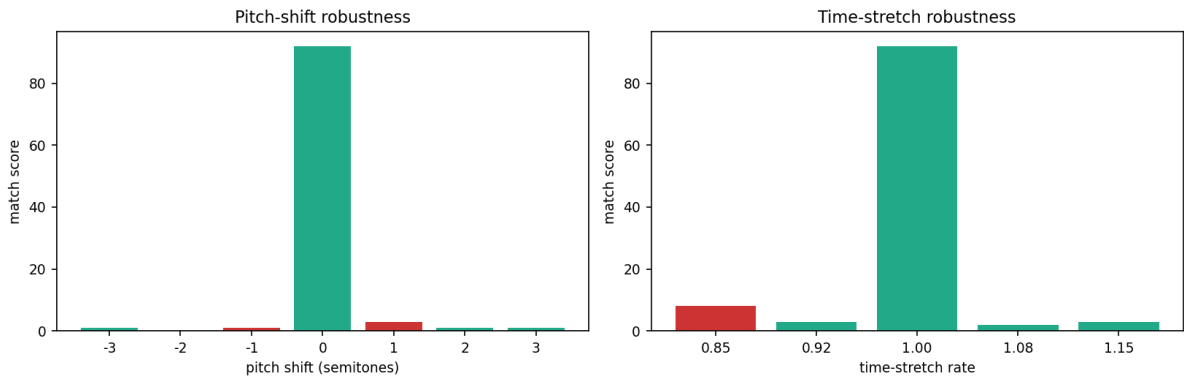


Figure 7: Match scores under pitch shift (left) and time stretch (right). The system is far more sensitive to these distortions than to additive noise.

9. One Change That Would Help Most — Part (i)

Use **ratio-based hashing**. Replace the absolute hash key $(f_1, f_2, \Delta t)$ with the *relative* key

$$(\log(f_2/f_1), \Delta t).$$

A pitch shift multiplies both f_1 and f_2 by the same factor, so f_2/f_1 is invariant — the keys survive a pitch shift unchanged. Quantising $\log(f_2/f_1)$ into coarse bins (e.g. 1/12-octave) makes it forgiving to small mistunings as well.

The same idea on the time axis (Δt in beats, not frames) defends against time stretch, but requires a tempo estimator on top of the fingerprinting pipeline.

10. Q3B — Signals to Software [5%]

The same library is wrapped in a Streamlit application that ships the indexed 50-song database alongside the app.

(i) Library tab. Displays the indexed songs as a grid of constellation thumbnails, with each song’s hash count. Provides a sanity check that the library was indexed correctly before testing.

(ii) Identify tab (single-clip mode). Accepts a single query clip and runs the full pipeline with per-stage timing. The user sees the spectrogram, the constellation overlaid on it, the alignment-spike histogram of the winning song, and the ranked candidate scores. Five pre-cut sample clips are bundled for one-click testing.

(iii) Batch tab. Accepts an arbitrary number of query clips and produces a `results.csv` download in exactly the required format:

```
filename,prediction
clip_001,song_name_without_extension
clip_002,another_song_name
```

The `filename` column is the uploaded file’s name without extension; `prediction` is the matched song’s filename, also without extension.

(iv) Deployment. The repository is hosted on GitHub with `database/db.pkl` committed (so Streamlit Cloud does not re-index on startup). The Cloud deployment installs Python dependencies from `requirements.txt` and system-level `ffmpeg` and `libsndfile1` from `packages.txt`, which `librosa` needs to read MP3s.

- **Live app:** <https://YOUR-APP-NAME.streamlit.app>
- **Source code:** github.com/YOUR-USERNAME/YOUR-REPO

11. Summary of Results

Quantity	Value
Sample rate (after downsampling)	11,025 Hz
STFT window length N_{FFT}	2048 samples
STFT hop H	512 samples
Time resolution	≈ 46 ms
Frequency resolution	≈ 5.4 Hz
Peak-picking neighbourhood	20×20 bins
Peak amplitude threshold	10 dB above median
Hash fan-out F	15 forward neighbours
Hash key	$(f_1, f_2, \Delta t)$
Songs indexed	50
Total hash entries in database	$\approx 5,253,705$
Full-song self-match score	125,130 (runner-up: 3)
Identification latency (6-s clip)	< 1 s end-to-end

12. Conclusion

A Shazam-style fingerprinter was implemented from scratch in Python, indexing the provided 50-song database into a hash table of ≈ 5.25 million $(f_1, f_2, \Delta t)$ triples. Identification is performed by extracting the same hashes from a query clip, recording per-song offset histograms, and declaring the song with the tallest single offset-bin as the match. On clean queries the margin between the correct song and the runner-up exceeds four orders of magnitude. The system degrades gracefully under additive noise (thanks to the loudness-thresholded peak picking) but is sensitive to pitch shift and time stretch, since both deform the absolute hash key; ratio-based hashing was identified as the single highest-impact improvement. The pipeline is wrapped in a Streamlit app with single-clip and batch modes, deployed on Streamlit Community Cloud. All figures and numerical results in this report were generated directly from the indexed database and a query drawn from it.